Solve Any Data Analysis Problem

Chapter 4 - Project 3: What is the "best" product?

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Exploration

In [2]:
```
november = pd.read_csv("./data/november.csv.gz")
december = pd.read_csv("./data/december.csv.gz")
```

In [3]:
```
events = pd.concat([november, december], axis=0, ignore_index=True)
print(events.shape)

events["event_time"] = pd.to_datetime(events["event_time"], format="%Y-%m-%d %H:%M:

events.head()
```

```
(7033125, 9)
```

Out[3]:

	event_time	event_type	product_id	category_id	category_code	
0	2019-11-01 00:00:14+00:00	cart	1005014	2053013555631882655	electronics.smartphone	sar
1	2019-11-01 00:00:41+00:00	purchase	13200605	2053013557192163841	furniture.bedroom.bed	
2	2019-11-01 00:01:04+00:00	purchase	1005161	2053013555631882655	electronics.smartphone	›
3	2019-11-01 00:03:24+00:00	cart	1801881	2053013554415534427	electronics.video.tv	sar
4	2019-11-01 00:03:39+00:00	cart	1005115	2053013555631882655	electronics.smartphone	

Sanity checks

In [4]:
```
events.isnull().sum()
```

Out[4]:
```
event_time            0
event_type            0
product_id            0
category_id           0
category_code         0
brand            395108
price                 0
user_id               0
user_session         27
dtype: int64
```

In [5]:
```
events["event_time"].agg(["min", "max"])
```

Out[5]:
```
min   2019-11-01 00:00:14+00:00
max   2019-12-31 23:59:09+00:00
Name: event_time, dtype: datetime64[ns, UTC]
```

In [6]:
```
events["event_type"].value_counts(dropna=False)
```

Out[6]:
```
cart        5276372
purchase    1756753
Name: event_type, dtype: int64
```

In [7]:
```
events["category_id"].nunique()
```

Out[7]:
```
862
```

In [8]:
```
events["category_code"].nunique()
```

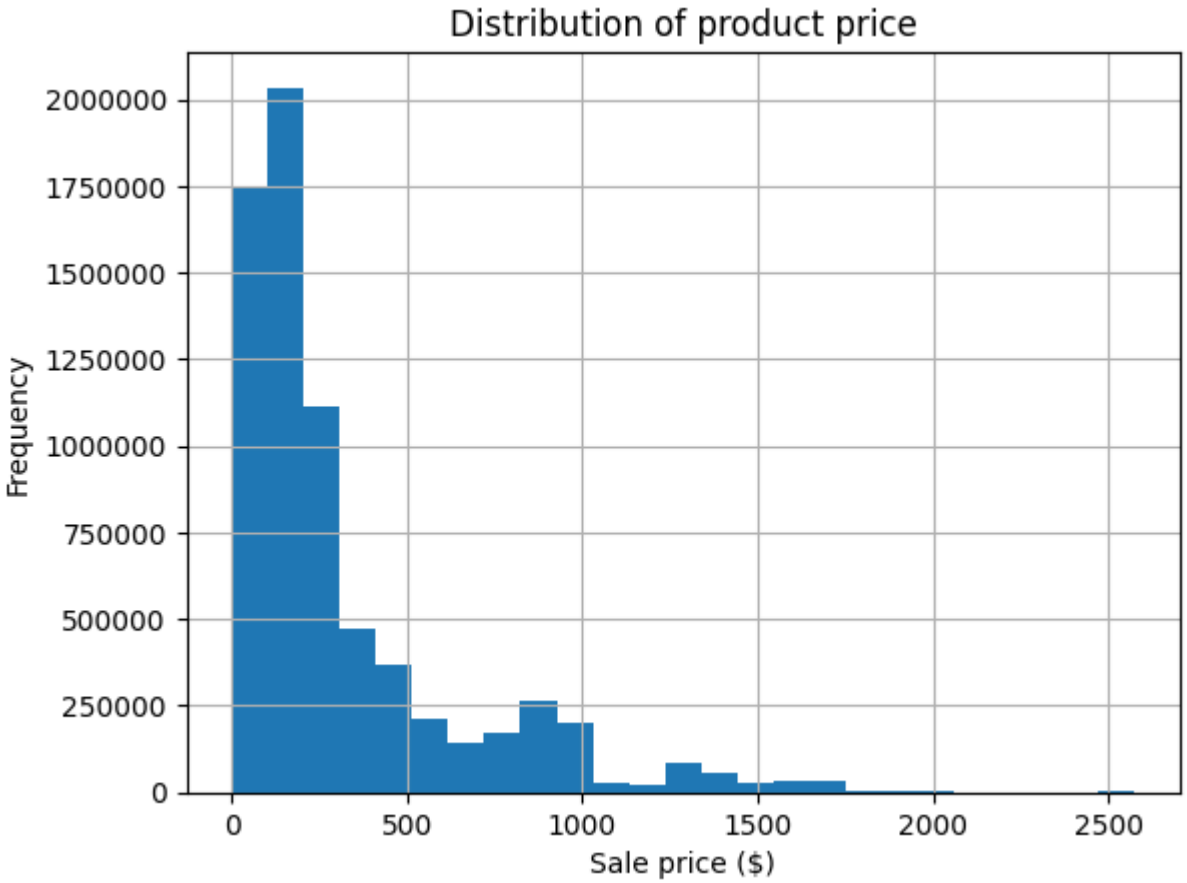
Out[8]:
```
134
```

In [9]:
```
events["brand"].value_counts().head(10)
```

Out[9]:
```
samsung    1756284
apple      1368364
xiaomi      722125
huawei      263155
oppo        128661
lg          123581
sony         78615
artel        72124
lucente      69998
lenovo       59700
Name: brand, dtype: int64
```

In [10]:
```
fig, axis = plt.subplots()

(
    events["price"]
    .hist(bins=25, ax=axis)
)

# disable scientific notation
```

```
axis.ticklabel_format(useOffset=False, style='plain')

axis.set(title="Distribution of product price",
         xlabel="Sale price ($)",
         ylabel="Frequency")

plt.show()
```



In [11]:
```
events["user_id"].nunique()
```

Out[11]:
```
1333980
```

Do product IDs match to the same category every time?

In [12]:
```
(
    events
    .groupby("product_id")
    ["category_code"]
    .nunique()
    .loc[lambda x: x > 1]
    .sort_values(ascending=False)
)
```

Out[12]:
```
product_id
1000978      2
13900087     2
13900049     2
13900054     2
13900055     2
             ..
5701183      2
5701186      2
5701190      2
5701192      2
100028003    2
Name: category_code, Length: 12884, dtype: int64
```

That's nearly 13,000 product IDs attached to multiple codes!

In [13]:
```
events["category_code"].value_counts().head(10)
```

Out[13]:
```
construction.tools.light            1823398
electronics.smartphone              1577185
electronics.audio.headphone          240308
electronics.clocks                   233198
sport.bicycle                        231644
apparel.shoes                        212586
appliances.kitchen.refrigerators     190523
appliances.personal.massager         185601
appliances.environment.vacuum        170290
appliances.kitchen.washer            170121
Name: category_code, dtype: int64
```

There are a lot of construction tools, which seems... odd.

In [14]:
```
events[events["category_code"] == "construction.tools.light"].sample(10, random_sta
```

Out[14]:

	event_time	event_type	product_id	category_id	category_co
2956496	2019-12-01 14:19:49+00:00	cart	1005284	2232732093077520756	construction.tools.lig
6467629	2019-12-28 14:09:22+00:00	cart	1004857	2232732093077520756	construction.tools.lig
4173392	2019-12-14 11:19:41+00:00	cart	1003910	2232732093077520756	construction.tools.lig
5065153	2019-12-19 14:40:21+00:00	cart	1005160	2232732093077520756	construction.tools.lig
4576741	2019-12-16 21:29:41+00:00	cart	1005100	2232732093077520756	construction.tools.lig
4283065	2019-12-15 11:59:48+00:00	cart	1005211	2232732093077520756	construction.tools.lig
3741431	2019-12-10 05:08:09+00:00	cart	1002544	2232732093077520756	construction.tools.lig
6981201	2019-12-31 10:55:21+00:00	cart	1005141	2232732093077520756	construction.tools.lig
6626408	2019-12-29 12:17:22+00:00	cart	1004858	2232732093077520756	construction.tools.lig
3057975	2019-12-02 14:50:52+00:00	cart	1003712	2232732093077520756	construction.tools.lig

These are probably smartphones?

In [15]:
```
(
    events
    .loc[events["category_code"] == "construction.tools.light", "brand"]
```

```
    .value_counts()
    .head(10)
)
```

Out[15]:
```
samsung    727599
apple      521481
xiaomi     304546
huawei     149347
oppo        68446
vivo        10709
meizu        9093
honor        4389
nokia        3231
omron        3003
Name: brand, dtype: int64
```

We don't have a lot of information, but most of these are phone manufacturers so probably miscategorised.

What we could do is in cases where a product ID has two categories, take the **other** one and use that everywhere.

In [16]:
```
dupe_product_ids = (
    events
    .groupby("product_id")
    ["category_code"]
    .nunique()
    .loc[lambda x: x > 1]
    .index
    .values
)

dupe_product_ids[:10]
```

Out[16]:
```
array([1000978, 1001588, 1001618, 1001619, 1002098, 1002100, 1002101,
       1002225, 1002367, 1002482], dtype=int64)
```

In [17]:
```
(
    events.loc[events["product_id"] == 1001588,
               "category_code"]
    .value_counts()
)
```

Out[17]:
```
construction.tools.light    50
electronics.smartphone      22
Name: category_code, dtype: int64
```

In [18]:
```
def get_correct_category_code(product_id_rows):
    # product_id_rows is all rows for a given product ID
    # so we can find the two categories associated with this ID
    categories = product_id_rows["category_code"].value_counts()

    # if one is construction.tools.light, return the other one
    if "construction.tools.light" in categories.index:
        return categories.index.drop("construction.tools.light").values[0]
```

```
    # otherwise return the majority category
    else:
        return categories.index[0]

corrected_categories = (
    events[events["product_id"].isin(dupe_product_ids)]
    .groupby("product_id")
    .apply(get_correct_category_code)
    .reset_index(name="corrected_category")
)

corrected_categories.head()
```

Out[18]:

	product_id	corrected_category
0	1000978	electronics.smartphone
1	1001588	electronics.smartphone
2	1001618	electronics.smartphone
3	1001619	electronics.smartphone
4	1002098	electronics.smartphone

In [19]:
```
corrected_categories["corrected_category"].value_counts()
```

Out[19]:
```
sport.bicycle                      658
electronics.smartphone             641
apparel.shoes                      498
appliances.personal.massager       455
electronics.audio.headphone        388
                                  ...
appliances.personal.hair_cutter      2
appliances.kitchen.washer            2
appliances.kitchen.fryer             2
apparel.belt                         1
electronics.audio.subwoofer          1
Name: corrected_category, Length: 126, dtype: int64
```

Ensure product_id is unique

In [20]:
```
corrected_categories["product_id"].value_counts().loc[lambda x: x>1]
```

Out[20]:
```
Series([], Name: product_id, dtype: int64)
```

Join this to the original data and overwrite the subcategory where this column doesn't match the original subcategory

In [21]:
```
events = events.merge(corrected_categories, on="product_id", how="left")
events.loc[events["corrected_category"].notnull(), "category_code"] = \
    events.loc[events["corrected_category"].notnull(), "corrected_category"]

events["category_code"].value_counts()
```

Out[21]:
```
electronics.smartphone              3350680
sport.bicycle                        384120
appliances.personal.massager         254300
electronics.clocks                   228546
appliances.kitchen.refrigerators     214637
                                     ...
apparel.skirt                           261
construction.tools.soldering            200
sport.diving                            144
computers.components.sound_card         118
auto.accessories.light                   41
Name: category_code, Length: 134, dtype: int64
```

That's more like it!!

In [22]:
```
events.drop(columns=["corrected_category"], inplace=True)
```

Let's now extract the "parent" category from the category_code

In [23]:
```
events["category"] = events["category_code"].str.split(".").str[0]

events = events.rename(columns={"category_code": "subcategory"})

events.head()
```

Out[23]:

	event_time	event_type	product_id	category_id	subcategor
0	2019-11-01 00:00:14+00:00	cart	1005014	2053013555631882655	electronics.smartphon
1	2019-11-01 00:00:41+00:00	purchase	13200605	2053013557192163841	furniture.bedroom.bed
2	2019-11-01 00:01:04+00:00	purchase	1005161	2053013555631882655	electronics.smartphon
3	2019-11-01 00:03:24+00:00	cart	1801881	2053013554415534427	appliances.personal.massage
4	2019-11-01 00:03:39+00:00	cart	1005115	2053013555631882655	electronics.smartphon

In [24]:
```
events["category"].value_counts()
```

Out[24]:
```
electronics     3908916
appliances      1250149
apparel          540955
sport            439029
computers        291790
furniture        216264
construction     144637
kids              98236
auto              89079
accessories       39350
country_yard       6106
medicine           5616
stationery         2998
Name: category, dtype: int64
```

Create a "product name" column with more information than just a product ID.

We've already fixed incorrect product categories, but let's check that a product ID only ever matches a single combination of category + brand

In [25]:
```
(
    events
    .assign(brand=events["brand"].fillna("No brand"))
    .groupby("product_id")
    ["brand"]
    .nunique()
    .loc[lambda x: x>1]
)
```

Out[25]:
```
product_id
1001618      2
1002310      2
1002786      2
1002877      2
1003080      2
            ..
100062901    2
100063013    2
100063159    2
100063161    2
100063351    2
Name: brand, Length: 1245, dtype: int64
```

In [26]:
```
events.loc[events["product_id"] == 1001618, "brand"].value_counts(dropna=False)
```

Out[26]:
```
apple    118
NaN       11
Name: brand, dtype: int64
```

In [27]:
```
events.loc[events["product_id"] == 1480638, "brand"].value_counts(dropna=False)
```

Out[27]:
```
asus      21
pulser    16
Name: brand, dtype: int64
```

Let's do the same "fix" for brands.

- for each product ID calculate the majority (non-null) brand
- fill NA values with that majority value
- overwrite other brands with majority value

In [28]:
```
duplicated_brands = (
    events
    .assign(brand = events["brand"].fillna("No brand")) # nunique doesn't count "Na
    .groupby("product_id")
    ["brand"]
    .nunique()
    .loc[lambda x: x > 1]
    .index
)

print(len(duplicated_brands))

duplicated_brands[:10]
```

```
1245
```

Out[28]:
```
Int64Index([1001618, 1002310, 1002786, 1002877, 1003080, 1003224, 1003238,
            1003330, 1003604, 1003851],
           dtype='int64', name='product_id')
```

In [29]:
```
def get_correct_brand(product_id_rows):
    # product_id_rows is all rows for a given product ID
    # so we can find the brands associated with this ID
    # value_counts will give us only non-NA values
    brand_counts = product_id_rows["brand"].value_counts(dropna=False)

    if isinstance(brand_counts.index[0], str):
        # no NULLs, just return the majority brand
        return brand_counts.index[0]

    # now if np.NaN is the only value, return it
    if len(brand_counts) == 1:
        return np.nan

    # otherwise return the second value (the majority non-null value)
    return brand_counts.index[1]

corrected_brands = (
    events[events["product_id"].isin(duplicated_brands)]
    .groupby("product_id")
    .apply(get_correct_brand)
    .reset_index(name="corrected_brand")
)

corrected_brands.head()
```

Out[29]:

	product_id	corrected_brand
0	1001618	apple
1	1002310	lg
2	1002786	apple
3	1002877	samsung
4	1003080	huawei

In [30]:
```
corrected_brands.isnull().sum()
```

Out[30]:
```
product_id         0
corrected_brand    0
dtype: int64
```

Join this to the original data and overwrite the brand where this column doesn't match the original brand

In [31]:
```
events = events.merge(corrected_brands, on="product_id", how="left")
events.loc[events["corrected_brand"].notnull(), "brand"] = \
    events.loc[events["corrected_brand"].notnull(), "corrected_brand"]
```

Check if brand is now unique among product IDs

In [32]:
```
events.groupby("product_id")["brand"].nunique().loc[lambda x: x>1]
```

Out[32]:
```
Series([], Name: brand, dtype: int64)
```

In [33]:
```
events.drop(columns=["corrected_brand"], inplace=True)
```

Now verify that a product ID matches only ONE brand + category combo (let's not try to fix category ID and just ignore it)

In [34]:
```
assert (
    len(events[["product_id", "category", "subcategory", "brand"]]
        .drop_duplicates())
    ==
    events["product_id"].nunique()
)
```

Finally create the product name (again, should be unique for each product ID)

In [36]:
```
def get_product_name(row):
    brand = ""

    # only include brand if it's available
    if isinstance(row["brand"], str):
        brand = row["brand"]

    return f"{str(row['product_id'])} - {brand} {row['subcategory']}"
```

```
events["product_name"] = events.apply(get_product_name, axis=1)

events.head()
```

Out[36]:

	event_time	event_type	product_id	category_id	subcategor
0	2019-11-01 00:00:14+00:00	cart	1005014	2053013555631882655	electronics.smartphon
1	2019-11-01 00:00:41+00:00	purchase	13200605	2053013557192163841	furniture.bedroom.be
2	2019-11-01 00:01:04+00:00	purchase	1005161	2053013555631882655	electronics.smartphon
3	2019-11-01 00:03:24+00:00	cart	1801881	2053013554415534427	appliances.personal.massage
4	2019-11-01 00:03:39+00:00	cart	1005115	2053013555631882655	electronics.smartphon

In [37]:
```
events.groupby("product_id")["product_name"].nunique().loc[lambda x: x > 1]
```

Out[37]:
```
Series([], Name: product_name, dtype: int64)
```

Save the modified data for later!

The above operations take a while so save a modified copy we can return to later.

We can then start the notebook from here next time to avoid waiting.

In [38]:
```
events.to_parquet("./data/events.parquet.gz", compression="gzip")
```

Part 2 (reading in pre-processed data) starts here

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
events = pd.read_parquet("./data/events.parquet.gz")
```

Split out by purchases and "cart" events too

In [3]:
```
events["event_type"].value_counts()
```

Out[3]:
```
cart        5276372
purchase    1756753
Name: event_type, dtype: int64
```

In [4]:
```
purchases = events[events["event_type"] == "purchase"].copy()
carts = events[events["event_type"] == "cart"].copy()
```

In [5]:
```
assert len(purchases) + len(carts) == len(events)
```

Sanity checks

Do we have any gaps?

In [ ]:
```
(
    events
    .assign(month=events["event_time"].dt.month,
            day=events["event_time"].dt.day)
    .groupby("month")
    ["day"]
    .nunique()
)
```

In [ ]:
```
fig, axis = plt.subplots(figsize=(6, 10))

(
    events
    .assign(month=events["event_time"].dt.month,
            day=events["event_time"].dt.day)
    [["month", "day"]]
    .value_counts()
    .sort_values()
    .plot
    .barh(ax=axis)
)

axis.set(title="Number of events per calendar day",
         xlabel="Number of rows",
         ylabel="Calendar day")

plt.show()
```

There are definitely some days with much more data than others. With more context we'd know whether these are particular sale days or strange artifacts in the data. 17th November 2019 wasn't officially Black Friday but could have been a similar sale day for example.

Or it could be an inflated set of "cart" events? Let's narrow that to purchases only:

In [ ]:
```
fig, axis = plt.subplots(figsize=(6, 10))

(
    purchases
    .assign(month=events["event_time"].dt.month,
            day=events["event_time"].dt.day)
    [["month", "day"]]
    .value_counts()
    .sort_values()
    .plot
    .barh(ax=axis)
)
```

That's taken some of the noise away, but there was definitely something going on on the 17th November.

Top products in December

- by volume sold
- by total revenue
- by number of unique users purchased

Top sellers by price

In [ ]:
```
(
    purchases
    .groupby("product_name")
    ["price"]
    .sum()
    .sort_values(ascending=False)
    .head(20)
)
```

Top sellers by volume

In [ ]:
```
(
    purchases
    ["product_name"]
    .value_counts()
    .head(20)
)
```

Top sellers by number of unique user purchases

In [ ]:
```
(
    purchases
    .groupby("product_name")
    ["user_id"]
    .nunique()
    .sort_values(ascending=False)
```

```
    .head(20)
)
```

Build a "league table" of metrics we care about:

- by volume sold (monthly)
- by total revenue (monthly)
- by number of unique users purchased (monthly)
- changes for the same product in November (to see which is the "best" product during Christmas)

These metrics only need the purchases data, but for conversion we'll need "cart" events too so we'll join that metric on separately.

In [7]:
```
purchases_league_table = (
    purchases
    .assign(
        november_count=np.where(purchases["event_time"].dt.month==11, 1, 0),
        november_revenue=np.where(purchases["event_time"].dt.month==11, purchases["
        november_user_id=np.where(purchases["event_time"].dt.month==11, purchases["
        december_count=np.where(purchases["event_time"].dt.month==12, 1, 0),
        december_revenue=np.where(purchases["event_time"].dt.month==12, purchases["
        december_user_id=np.where(purchases["event_time"].dt.month==12, purchases["
    .groupby(["product_id", "product_name"])
    .agg(november_volume=('november_count', 'sum'),
         november_revenue=('november_revenue', 'sum'),
         november_users=('november_user_id', 'nunique'),
         december_volume=('december_count', 'sum'),
         december_revenue=('december_revenue', 'sum'),
         december_users=('december_user_id', 'nunique')
    )
    .assign(
        volume_diff=lambda x:
            x["december_volume"] - x["november_volume"],
        revenue_diff=lambda x:
            x["december_revenue"] - x["november_revenue"],
        users_diff=lambda x:
            x["december_users"] - x["november_users"])
    .reset_index()
)

purchases_league_table.head()
```

Out[7]:

	product_id	product_name	november_volume	november_revenue	november_use
0	1000978	1000978 - samsung electronics.smartphone	20	6135.32	1
1	1001588	1001588 - meizu electronics.smartphone	6	766.29	
2	1001605	1001605 - apple electronics.smartphone	0	0.00	
3	1001606	1001606 - apple electronics.smartphone	0	0.00	
4	1001618	1001618 - apple electronics.smartphone	36	18059.76	2

Conversion

In [8]:

```
conversion_table = (
    pd.pivot_table(
        data=events.assign(month=events["event_time"].dt.month),
        index=["product_id", "product_name"],
        columns=["month", "event_type"],
        values="user_session",
        aggfunc="count"
    )
    .fillna(0)
    .set_axis(labels=["november_cart", "november_sold", "december_cart", "december_
              axis=1)
    .reset_index()
    .assign(november_conversion=lambda x: x["november_sold"] / x["november_cart"],
            december_conversion=lambda x: x["december_sold"] / x["december_cart"])
)

conversion_table.head()
```

Out[8]:

	product_id	product_name	november_cart	november_sold	december_cart	dece
0	1000894	1000894 - texet electronics.smartphone	0.0	0.0	4.0	
1	1000978	1000978 - samsung electronics.smartphone	60.0	20.0	58.0	
2	1001588	1001588 - meizu electronics.smartphone	16.0	6.0	37.0	
3	1001605	1001605 - apple electronics.smartphone	0.0	0.0	42.0	
4	1001606	1001606 - apple electronics.smartphone	0.0	0.0	23.0	

In [9]:

```
league_table = purchases_league_table.merge(conversion_table, on=["product_id", "pr
league_table.head()
```

Out[9]:

	product_id	product_name	november_volume	november_revenue	november_use
0	1000978	1000978 - samsung electronics.smartphone	20	6135.32	1
1	1001588	1001588 - meizu electronics.smartphone	6	766.29	
2	1001605	1001605 - apple electronics.smartphone	0	0.00	
3	1001606	1001606 - apple electronics.smartphone	0	0.00	
4	1001618	1001618 - apple electronics.smartphone	36	18059.76	2

In [12]:
```
league_table.dtypes
```

Out[12]:
```
product_id              int64
product_name           object
november_volume         int32
november_revenue      float64
november_users          int64
december_volume         int32
december_revenue      float64
december_users          int64
volume_diff             int32
revenue_diff          float64
users_diff              int64
november_cart         float64
november_sold         float64
december_cart         float64
december_sold         float64
november_conversion   float64
december_conversion   float64
dtype: object
```

Again check that product ID is still unique

In [10]:
```
assert len(league_table) == purchases["product_id"].nunique()
```

So now we have a bunch of metrics in place. How do we "score" each product?

Depends what we care about!

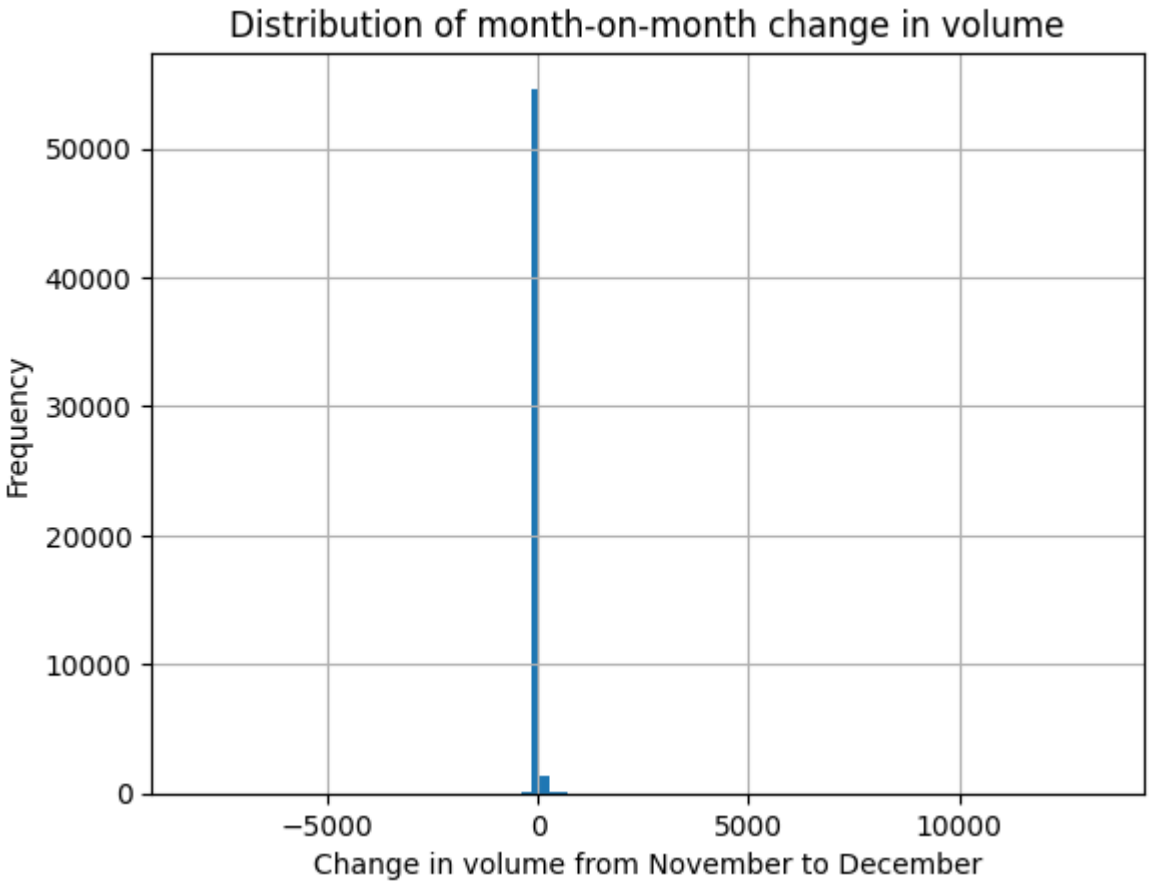
- we could pick the one metric to sort the list by and just think of the top N products as "best"
- we could weight each metric by its relative importance to others and create a "weighted combination" of metric values
- if we care about best Christmas performers we could look at the distribution of one of the "diff" columns and see the positive outliers

Lots of options! As the brief was about the Christmas period, let's go for that latter option.

Which metric do we care about most?

- Which product brought in more revenue? Probably not, seeing as more expensive items will skew the figures
- Number of unique users that bought the product? Possibly
- Volume of sold items seems like a solid start
- Conversion is then something to look at afterwards

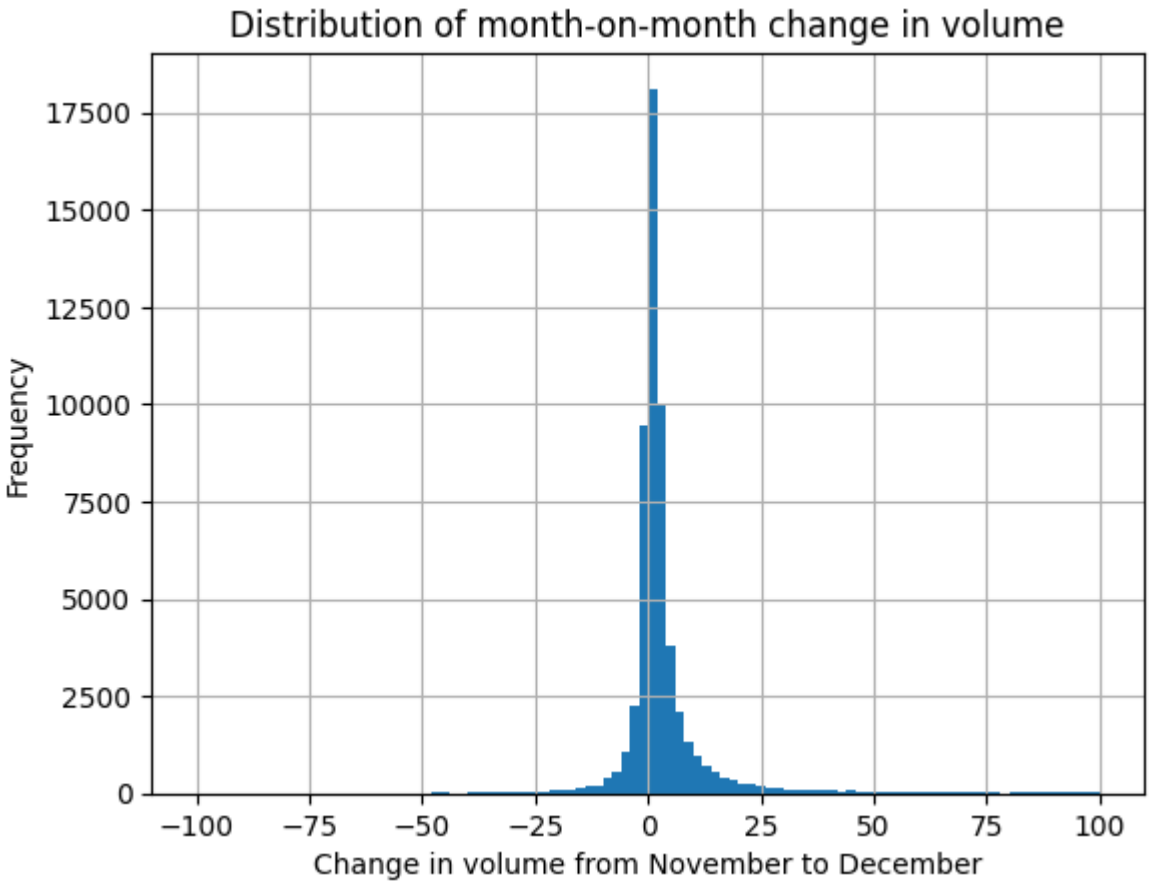
In [14]:
```
fig, axis = plt.subplots()

league_table["volume_diff"].hist(bins=100, ax=axis)

axis.set(title="Distribution of month-on-month change in volume",
         xlabel="Change in volume from November to December",
         ylabel="Frequency")

plt.show()
```



Most items haven't changed their behaviour much, let's zoom in to check

In [15]:
```
fig, axis = plt.subplots()

league_table.loc[league_table["volume_diff"].between(-100,100), "volume_diff"].hist

axis.set(title="Distribution of month-on-month change in volume",
         xlabel="Change in volume from November to December",
```

```
        ylabel="Frequency")

plt.show()
```



We really should be looking at percentage change so large-volume items don't skew the data

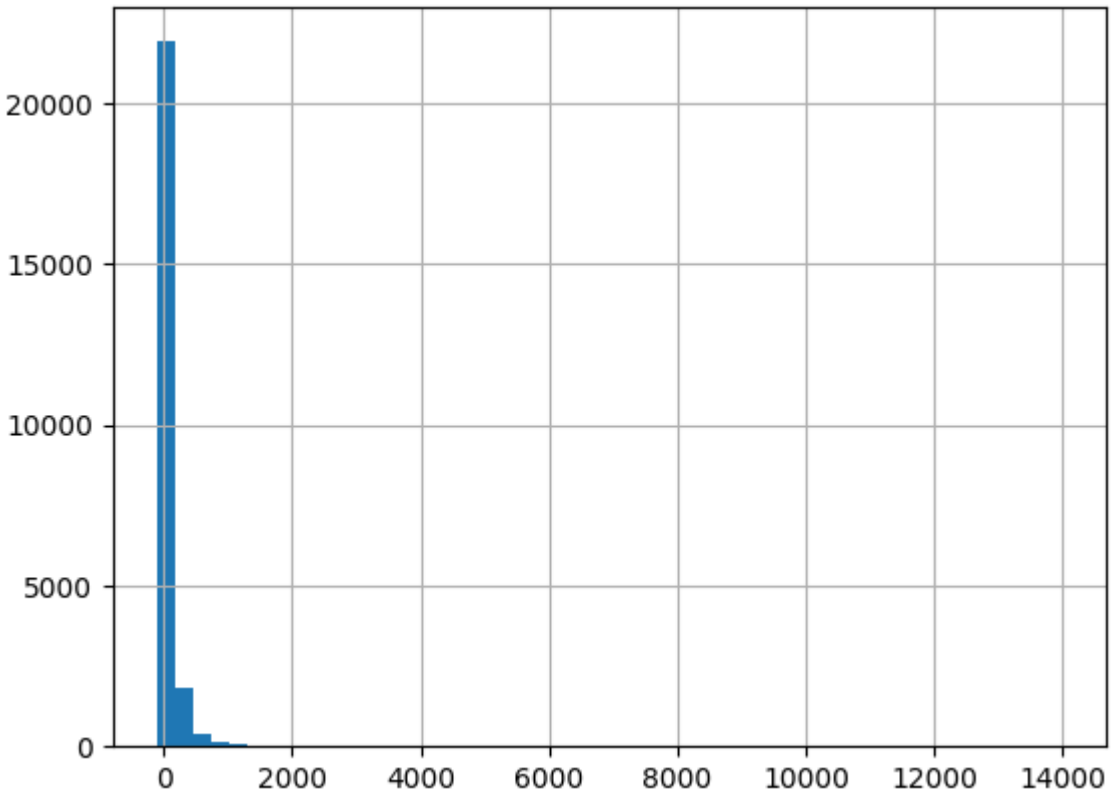
In [ ]:
```
league_table.head()
```

In [22]:
```
league_table["volume_diff_pct"] = (
    100 * (league_table["volume_diff"] / league_table["november_volume"])
)

(
    league_table
    .replace([np.inf, -np.inf], np.nan)
    .dropna()
    .sort_values("volume_diff_pct", ascending=False).head()
)
```

Out[22]:

	product_id	product_name	november_volume	november_revenue	no
55	1002995	1002995 - apple electronics.smartphone	3	849.36	
5523	3100883	3100883 - scarlett appliances.kitchen.blender	1	20.57	
52266	100023495	100023495 - lenovo electronics.smartphone	1	617.52	
72	1003141	1003141 - apple appliances.kitchen.refrigerators	1	382.97	
304	1004157	1004157 - samsung electronics.smartphone	1	756.78	

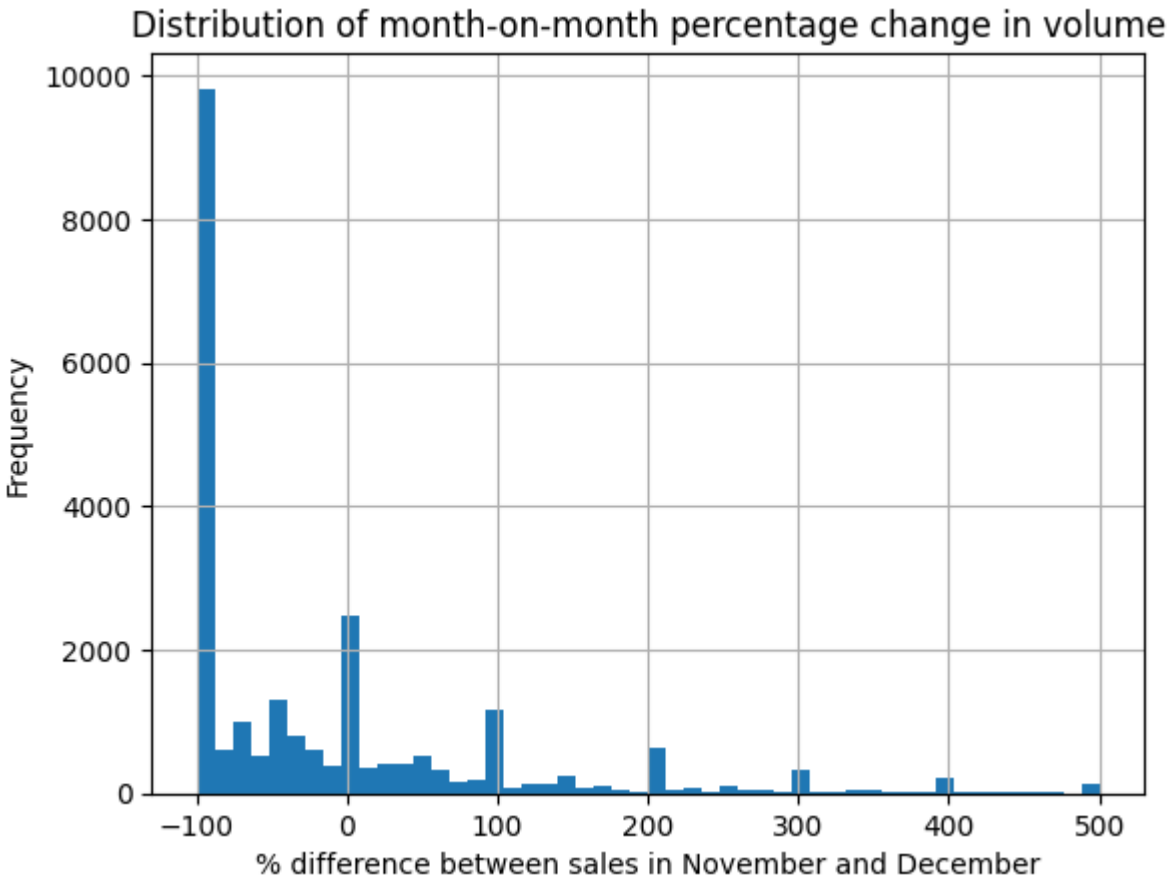
In [17]:
```
(
    league_table["volume_diff_pct"]
    .replace([np.inf, -np.inf], np.nan)
    .dropna()
    .hist(bins=50)
);
```



Let's zoom in again

In [24]:
```
fig, axis = plt.subplots()

(
    league_table["volume_diff_pct"]
    .replace([np.inf, -np.inf], np.nan)
```

```
    .dropna()
    .loc[lambda x: x.between(-101,501)]
    .hist(bins=50, ax=axis)
)

axis.set(
    title="Distribution of month-on-month percentage change in volume",
    xlabel="% difference between sales in November and December",
    ylabel="Frequency"
)

plt.show()
```



Let's look at the biggest differences (excluding when we didn't sell any in November)

In [ ]:
```
(
    league_table[(league_table["volume_diff_pct"] > 200)
                 & (np.isinf(league_table["volume_diff_pct"]) == False)]
    .sort_values("volume_diff_pct", ascending=False)
)
```

Now the cases where we sold none in November, but some in December

In [ ]:
```
(
    league_table[np.isinf(league_table["volume_diff_pct"])]
    .sort_values("december_volume", ascending=False)
)
```

So we have:

- products that sold much more in December than November
- products that sold nothing in November but at least 1 in December

What do these kinds of products have in common? Let's extract them and look at the distribution of product category.

We can either choose all products in those two categories, or have some sensible cut-offs:

- let's say anything that sold more than double in December vs. November, and at least 10 in November, gets counted
- and anything where we sold at least 100 in December but 0 in November

Let's also extract a unique product lookup table to re-join to these metrics

We won't include the product price because that changes depending on discounts etc.:

In [ ]:
```
events.groupby("product_name")["price"].nunique().loc[lambda x: x>1].sort_values(as
```

In [ ]:
```
events.loc[events["product_name"] == "1004872 - samsung electronics.smartphone", "p
```

This particular Samsung phone has had 185 different prices! Possibly due to discounts, vouchers/coupons, differing exchange rates (if figures are converted to USD from another currency), age/popularity of the product etc.

In [ ]:
```
events.head(1)
```

Product ID should be unique in this product catalog, and sometimes the same product has multiple category IDs for a start...

In [26]:
```
events.groupby(["product_id", "product_name", "subcategory", "brand", "category"])[
```

Out[26]:
```
product_id  product_name                                          subcategory
                    brand         category
1000978     1000978 - samsung electronics.smartphone            electronics.sm
artphone            samsung       electronics    2
1001588     1001588 - meizu electronics.smartphone              electronics.sm
artphone            meizu         electronics    2
1001618     1001618 - apple electronics.smartphone              electronics.sm
artphone            apple         electronics    2
1001619     1001619 - apple electronics.smartphone              electronics.sm
artphone            apple         electronics    2
1002098     1002098 - samsung electronics.smartphone            electronics.sm
artphone            samsung       electronics    2

                                                ..
100027045   100027045 - clatronic apparel.shoes.sandals         apparel.shoes.
sandals             clatronic     apparel        2
100027452   100027452 - pulser appliances.kitchen.refrigerators appliances.kit
chen.refrigerators  pulser        appliances     2
100027645   100027645 - total appliances.kitchen.hood           appliances.kit
chen.hood           total         appliances     2
100027728   100027728 - schaublorenz appliances.kitchen.dishwasher  appliances.kit
chen.dishwasher     schaublorenz  appliances     2
100028003   100028003 - kivi appliances.personal.massager       appliances.per
sonal.massager      kivi          appliances     2
Name: category_id, Length: 17069, dtype: int64
```

So the product catalog should take this into account

In [27]:
```
product_catalog = (
    events[["product_id", "product_name", "category_id", "subcategory", "brand", "c
    .drop_duplicates(subset=["product_id", "product_name", "subcategory", "brand",
)

assert len(product_catalog) == events["product_id"].nunique()

print(product_catalog.shape)
product_catalog.head()
```

```
(83838, 6)
```

Out[27]:

	product_id	product_name	category_id	subcategor
0	1005014	1005014 - samsung electronics.smartphone	2053013555631882655	electronics.smartphon
1	13200605	13200605 - furniture.bedroom.bed	2053013557192163841	furniture.bedroom.be
2	1005161	1005161 - xiaomi electronics.smartphone	2053013555631882655	electronics.smartphon
3	1801881	1801881 - samsung appliances.personal.massager	2053013554415534427	appliances.personal.massage
4	1005115	1005115 - apple electronics.smartphone	2053013555631882655	electronics.smartphon

In [28]:
```
DEC_VS_NOV_PCT_CUTOFF = 200
NOV_VOLUME_CUTOFF = 10
ONLY_DEC_VOLUME_CUTOFF = 100

december_high_performers = (
    pd.concat(
    [
        league_table[(np.isinf(league_table["volume_diff_pct"]) == False)
            & (league_table["november_volume"] > NOV_VOLUME_CUTOFF)
            & (league_table["volume_diff_pct"] > DEC_VS_NOV_PCT_CUTOFF)],
        league_table[(np.isinf(league_table["volume_diff_pct"]))
            & (league_table["december_volume"] > ONLY_DEC_VOLUME_CUTOFF)]
    ],
    axis=0,
    ignore_index=True)
    .merge(product_catalog.drop(columns="product_name"), on="product_id") # no need
)

print(december_high_performers.shape)
december_high_performers.head()
```

```
(449, 22)
```

Out[28]:

	product_id	product_name	november_volume	november_revenue	november_use
0	1002527	1002527 - apple electronics.smartphone	14	10086.08	1
1	1003533	1003533 - samsung electronics.smartphone	123	53570.29	10
2	1003712	1003712 - samsung electronics.smartphone	519	309958.00	40
3	1003770	1003770 - huawei electronics.smartphone	11	3329.24	1
4	1003801	1003801 - apple electronics.smartphone	107	70989.71	8

5 rows × 22 columns

In [29]:
```
from IPython.display import display

for col in ["category", "subcategory", "brand"]:
    print(col)
    display(december_high_performers[col].value_counts())
    print("==============================\n")
```

```
category
```

```
apparel         121
appliances      102
electronics      85
computers        36
furniture        32
kids             24
construction     20
sport            13
auto              8
country_yard      4
medicine          2
accessories       1
stationery        1
Name: category, dtype: int64
==============================

subcategory
electronics.smartphone               43
apparel.shoes                        37
appliances.kitchen.coffee_grinder    33
apparel.shoes.sandals                28
appliances.personal.massager         26
                                     ..
electronics.audio.microphone          1
apparel.shoes.ballet_shoes            1
kids.dolls                            1
appliances.personal.hair_cutter       1
stationery.cartrige                   1
Name: subcategory, Length: 76, dtype: int64
==============================

brand
lucente    56
xiaomi     31
sony       24
samsung    20
huawei     18
           ..
babyzen     1
palit       1
galaxy      1
gorenje     1
saeshin     1
Name: brand, Length: 128, dtype: int64
==============================
```

In [30]:
```
december_high_performers[["category", "subcategory", "brand"]].value_counts().head(
```

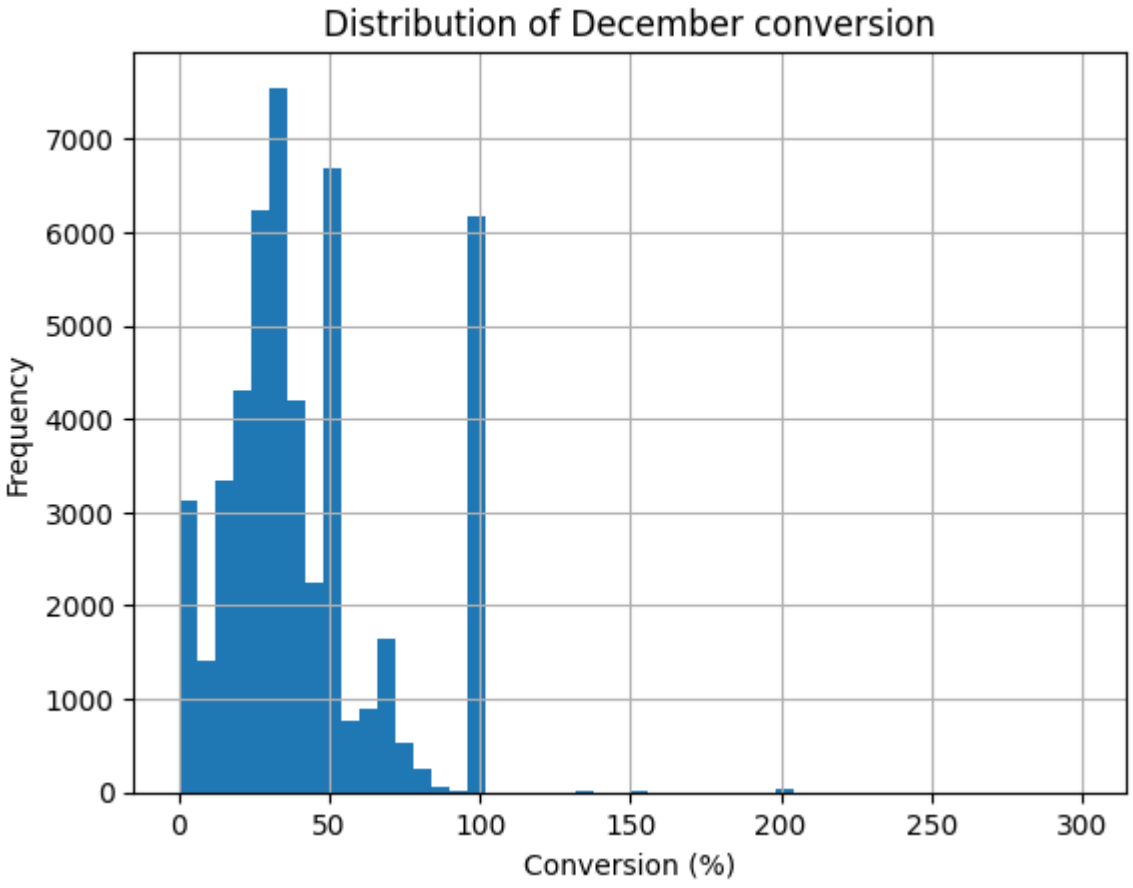
Out[30]:
```
category     subcategory                          brand
appliances   appliances.kitchen.coffee_grinder    lucente    33
electronics  electronics.smartphone               xiaomi     12
apparel      apparel.shoes                        sony       11
kids         kids.toys                            lucente    10
electronics  electronics.smartphone               huawei      9
                                                  samsung     8
apparel      apparel.shoes                        lg          7
computers    computers.peripherals.printer        lucente     7
furniture    furniture.kitchen.chair              xiaomi      6
electronics  electronics.smartphone               apple       6
dtype: int64
```

Looks like various shoes, smartphones, coffee grinders, and clocks all had upticks in December vs. November.

An alternative view

Let's now look at best products in terms of conversion.

In [38]:
```
fig, axis = plt.subplots()

(
    league_table[(np.isinf(league_table["december_conversion"]) == False)
                 & (np.isnan(league_table["december_conversion"]) == False)]
    ["december_conversion"]
    .mul(100)
    .hist(bins=50, ax=axis)
)

axis.set(
    title="Distribution of December conversion",
    xlabel="Conversion (%)",
    ylabel="Frequency"
)

plt.show()
```



Some fun data issues like >100% conversion aside (we're either missing cart events or they happened in November if the purchase happened on the 1st December just after midnight) it looks like quite a high conversion rate from cart -> purchase, which makes sense.

Let's look at the top performers (data errors included):

In [32]:
```
(
    league_table[(np.isinf(league_table["december_conversion"]) == False)
                 & (np.isnan(league_table["december_conversion"]) == False)
                 & (league_table["december_conversion"] > 0.99)]
    .sort_values("december_conversion", ascending=False)
    .head(20)
)
```

Out[32]:

	product_id	product_name	november_volume	november_revenue	n
17517	9200694	9200694 - apparel.scarf	0	0.00	
46637	44700039	44700039 - apparel.shirt	0	0.00	
32318	21410218	21410218 - orient electronics.clocks	4	646.60	
25735	17200764	17200764 - furniture.living_room.sofa	4	4931.92	
54792	100044324	100044324 - premont accessories.bag	0	0.00	
50955	100015261	100015261 - blackbox electronics.clocks	0	0.00	
17280	9101658	9101658 - asus furniture.kitchen.table	0	0.00	
37768	26403333	26403333 - computers.peripherals.printer	0	0.00	
15242	7300445	7300445 - kingston appliances.personal.massager	0	0.00	
54809	100044387	100044387 - sokolov sport.ski	0	0.00	
3799	2403060	2403060 - akpo appliances.personal.massager	0	0.00	
27317	17500969	17500969 - apparel.costume	0	0.00	
12633	6400170	6400170 - intel apparel.shoes.step_ins	0	0.00	
40453	28705382	28705382 - escan apparel.shoes.keds	0	0.00	
43859	30000105	30000105 - construction.tools.welding	0	0.00	
52827	100028739	100028739 - sokolov appliances.kitchen.fryer	0	0.00	
32998	21900005	21900005 - d-clinic construction.tools.pump	0	0.00	
17145	9101253	9101253 - computers.peripherals.mouse	2	19.52	
56020	100057304	100057304 - altair sport.bicycle	0	0.00	
17507	9200683	9200683 - hp computers.peripherals.keyboard	2	86.48	

Let's look at the top one of those:

In [33]:
```
events[(events["product_id"] == 9200694) & (events["event_time"].dt.month == 12)]
```

Out[33]:

	event_time	event_type	product_id	category_id	subcategory	bran
4698592	2019-12-17 13:01:28+00:00	cart	9200694	2232732104343421549	apparel.scarf	Non
4698610	2019-12-17 13:01:38+00:00	purchase	9200694	2232732104343421549	apparel.scarf	Non
5124731	2019-12-20 04:04:58+00:00	purchase	9200694	2232732104343421549	apparel.scarf	Non
5192399	2019-12-20 10:56:30+00:00	purchase	9200694	2232732104343421549	apparel.scarf	Non

That's the same user buying the same scarf 3 times after having it in their cart once. Possible theories:

- data error (missing cart events for 2 purchases)
- the platform allows users to re-purchase something they've already bought from, say, an "orders" page (hence no cart event)

Either way, an inflated conversion figure. Let's look at top performers under 100% conversion and with at least 2 users:

In [34]:

```
(
    league_table[(np.isinf(league_table["december_conversion"]) == False)
                 & (np.isnan(league_table["december_conversion"]) == False)
                 & (league_table["december_conversion"] <= 1)
                 & (league_table["december_users"] > 1)]
    .sort_values("december_conversion", ascending=False)
    .head(20)
)
```

Out[34]:

	product_id	product_name	november_volume	november_revenue	no
56281	100063282	100063282 - sokolov construction.components.fa...	0	0.00	
20756	12202338	12202338 - torrent sport.bicycle	4	463.24	
45276	36000005	36000005 - vinzer apparel.shoes	1	48.65	
20797	12202504	12202504 - trinx sport.bicycle	0	0.00	
45263	35900037	35900037 - wellberg apparel.shoes	0	0.00	
30191	21400406	21400406 - boccia electronics.clocks	0	0.00	
45220	35200685	35200685 - visavis apparel.shoes.moccasins	1	7.79	
20776	12202408	12202408 - mongoose sport.bicycle	0	0.00	
30258	21400634	21400634 - orient electronics.clocks	1	118.15	
30317	21400844	21400844 - casio electronics.clocks	0	0.00	
14056	7003889	7003889 - adamex kids.skates	0	0.00	
10186	5000731	5000731 - janome appliances.sewing_machine	0	0.00	
14065	7004023	7004023 - adamex kids.carriage	1	159.59	
53042	100029608	100029608 - sokolov sport.ski	0	0.00	
30295	21400744	21400744 - tissot electronics.clocks	0	0.00	
45023	34800428	34800428 - carfashion computers.peripherals.pr...	0	0.00	
45280	36000024	36000024 - rondell appliances.kitchen.kettle	6	169.74	
4406	2700920	2700920 - midea appliances.kitchen.refrigerators	14	12118.76	
30103	21400103	21400103 - casio electronics.clocks	0	0.00	
30082	21400039	21400039 - casio electronics.clocks	2	310.44	

Still quite low number of users and purchases, let's set some higher boundaries on those:

In [35]:
```
MIN_USERS = 5
MIN_PURCHASES = 10
CONVERSION_LOWER_LIMIT = 0.7

best_december_converters = (
    league_table[(np.isinf(league_table["december_conversion"]) == False)
             & (np.isnan(league_table["december_conversion"]) == False)
             & (league_table["december_conversion"].between(CONVERSION_LOWER_LIMIT,
             & (league_table["december_users"] > MIN_USERS)
             & (league_table["december_sold"] > MIN_PURCHASES)]
    .sort_values("december_conversion", ascending=False)
    .merge(product_catalog.drop(columns=["product_name"]), on=["product_id"]) # aga
)

print(best_december_converters.shape)
best_december_converters.head()
```

```
(55, 22)
```

Out[35]:

	product_id	product_name	november_volume	november_revenue	novem
0	26403438	26403438 - rusgoldart computers.peripherals.pr...	0	0.00	
1	14100671	14100671 - yamaha electronics.audio.acoustic	1	428.30	
2	26205367	26205367 - construction.components.faucet	0	0.00	
3	1307068	1307068 - lenovo computers.notebook	11	3338.28	
4	2200061	2200061 - sony furniture.bedroom.blanket	0	0.00	

5 rows × 22 columns

Those limits clear out 100% conversion figures, which leave us with the "best converters" (but only 55 products are above our arbitrary threshold of 70% conversion).

In [36]:
```
best_december_converters["category"].value_counts()
```

Out[36]:
```
furniture       9
construction    8
computers       7
appliances      7
electronics     6
kids            6
apparel         6
auto            3
sport           2
accessories     1
Name: category, dtype: int64
```

In [40]:
```
best_december_converters.loc[best_december_converters["category"].isin(["furniture"
```

Out[40]:
```
construction.components.faucet    6
furniture.bedroom.bed             3
construction.tools.drill          2
furniture.bathroom.bath           2
furniture.bedroom.blanket         1
furniture.kitchen.table           1
furniture.universal.light         1
furniture.living_room.sofa        1
Name: subcategory, dtype: int64
```

Which product is "best" and where do we go from here?

- "best" clearly doesn't have a single answer. All of the above answers would change if we changed our cutoff values for example
- we don't have enough data to compare beyond November. Maybe that's a quiet month and differences in volumes aren't just due to Christmas performance
- a more sophisticated method would be to forecast volumes for December and see which product outperformed its "expected" sales in December

Plenty of avenues to explore further after an initial discussion with our stakeholder!